

SWP Calculator

Project report for an interactive Systematic Withdrawal Plan calculator built to help users estimate corpus longevity, remaining balance, and the impact of inflation-adjusted withdrawals.

TECHNOLOGY

Next.js 16 + React 19

CORE PURPOSE

Investor education

MODES

Duration and Balance

PREPARED ON

15 March 2026

1. Project Summary

The SWP Calculator is a web-based financial planning tool that models monthly withdrawals from an investment corpus. It is designed to make withdrawal planning easier to understand through instant calculation, visual feedback, and a structured yearly breakdown.

The application focuses on clarity and educational value. Users can experiment with different corpus sizes, withdrawal amounts, expected returns, and inflation assumptions to see how quickly a corpus may deplete or how much balance may remain after a chosen period.

**SIMULATION
HORIZON**

40 Years

**CALCULATION
STYLE**

Monthly

RESULT VIEWS

KPIs + Chart

VALIDATION

Jest Tests

2. Objectives

- Estimate how long a withdrawal plan can sustain a starting corpus.
- Show the remaining corpus after a user-selected investment period.
- Explain the effect of return assumptions and inflation step-up on withdrawals.
- Present results in a format that is simple, visual, and suitable for investor awareness.

3. Core Features

Functional Features

- Interactive sliders for corpus, withdrawal, annual return, and target years.
- Dual operating modes: corpus duration and target-year balance.
- Optional inflation-adjusted step-up in annual withdrawals.
- Automatic depletion detection with alert messaging.
- Year-by-year aggregation of start balance, withdrawals, returns, and end balance.

User Experience Features

- Sticky header with clear mode switch.
- Live KPI cards for key outcomes.
- Corpus trend chart with cumulative withdrawal line.
- Indian-number formatting for easier reading.
- Assumptions and disclaimer sections for context.

4. Financial Logic

The main financial engine is implemented as pure TypeScript logic, which keeps it independent from the UI and easy to test.

$$\text{Monthly rate } r = \text{annualReturn} / 100 / 12$$

For each simulated month, the current balance first earns monthly return and then the withdrawal is applied. When inflation step-up is enabled, the withdrawal amount increases once every 12 months by the configured inflation percentage.

$$\text{balance} = \text{balance} + (\text{balance} \times r) - \text{withdrawal}$$

If the balance is insufficient to fund the next withdrawal, the engine records the exact depletion month, withdraws the remaining amount, and stops the simulation.

5. Input Model

Input	Description	Range / Default
Initial Corpus	Starting investment amount used for withdrawal simulation.	₹1,00,000 to ₹1,00,00,000, default ₹25,00,000
Monthly Withdrawal	Amount withdrawn from the corpus every month.	₹1,000 to ₹2,00,000, default ₹15,000
Expected Annual Return	Assumed annual return converted into a monthly growth rate.	0% to 20%, default 8%
Inflation Step-Up	Option to increase the withdrawal once each year.	Optional, default Off
Inflation Rate	Yearly percentage increase in withdrawal when step-up is enabled.	0% to 15%, default 5%
Mode	Select whether to calculate corpus duration or balance at a target year.	Duration / Balance
Target Years	Applicable in Balance mode for year-based projection.	1 to 40 years, default 10 years

6. Output Model

Main Outputs

- Corpus lasts for X years and months.
- Remaining corpus at selected target year.
- Final balance at end of simulation.
- Total withdrawn amount.
- Total returns earned.

Detailed Outputs

- Year-wise start and end balance.
- Annual withdrawals.
- Annual returns earned.
- Cumulative withdrawals.
- Depletion-year marker in table and warning banner.

7. Architecture

Layer	Responsibility	Key Files
UI Layer	Displays the calculator, chart, stats, and supporting content.	`app/page.tsx`, `components/*`
State Layer	Stores calculator inputs and recomputes results when values change.	`hooks/useSWP.ts`
Logic Layer	Runs month-wise SWP simulation and number formatting.	`lib/swpCalc.ts`

Layer	Responsibility	Key Files
Type Layer	Defines strongly typed inputs, outputs, and table rows.	`types/swp.ts`
Testing Layer	Verifies financial correctness and edge-case handling.	`__tests__/swpCalc.test.ts`

- Next.js 16
- React 19
- TypeScript
- Tailwind CSS 4
- Chart.js
- Jest

8. Testing And Validation

The project includes a focused Jest test suite for the financial engine. Because the calculation logic is isolated from React components, behavior can be validated independently and repeatably.

What Is Tested

- Annual-to-monthly rate conversion
- Month-one and month-two formula accuracy
- Depletion detection and stop conditions
- Inflation-based withdrawal step-up
- Balance mode accuracy for target years
- Totals and yearly-row aggregation
- Zero-return and low-corpus edge cases
- Known reference-value checks

Why It Matters

- Builds trust in financial outputs.
- Reduces regression risk during UI changes.
- Keeps the logic reusable for future extensions.
- Supports hackathon demonstrations with defensible results.

9. Strengths Of The Solution

- Clean separation between presentation, state, and financial logic.
- Fast feedback loop with instant updates as inputs change.
- Educational framing with assumptions and disclaimer sections.
- Accessible controls and clear warning states.
- Suitable for demonstration, extension, and future production hardening.

10. Limitations

- Uses a constant return assumption instead of variable market scenarios.
- Does not account for taxes, fees, or exit loads.
- Inflation is modeled as a simple yearly step-up rather than a dynamic curve.
- Simulation horizon is capped at 40 years.
- The current project emphasizes educational planning, not advisory decision-making.

11. Future Enhancements

- Add scenario comparison with side-by-side plans.
- Support SIP plus SWP combined retirement modeling.
- Include tax-aware and fee-aware calculations.
- Export results directly from the app.
- Add Monte Carlo or variable-return simulations.

12. Conclusion

The SWP Calculator demonstrates a practical and well-structured approach to modeling systematic withdrawals. It combines a polished user interface with a testable financial engine, making it useful both as a project showcase and as an educational planning tool.

This project is for information and awareness purposes only. Actual investment outcomes depend on market conditions and many real-world factors that are not part of this simulation.

Source files: ``app/page.tsx``, ``hooks/useSWP.ts``, ``lib/swpCalc.ts``, ``components/*``, ``__tests__/swpCalc.test.ts``